

# A FORMAL LEARNING MODEL

Dr. Krishna Bathula

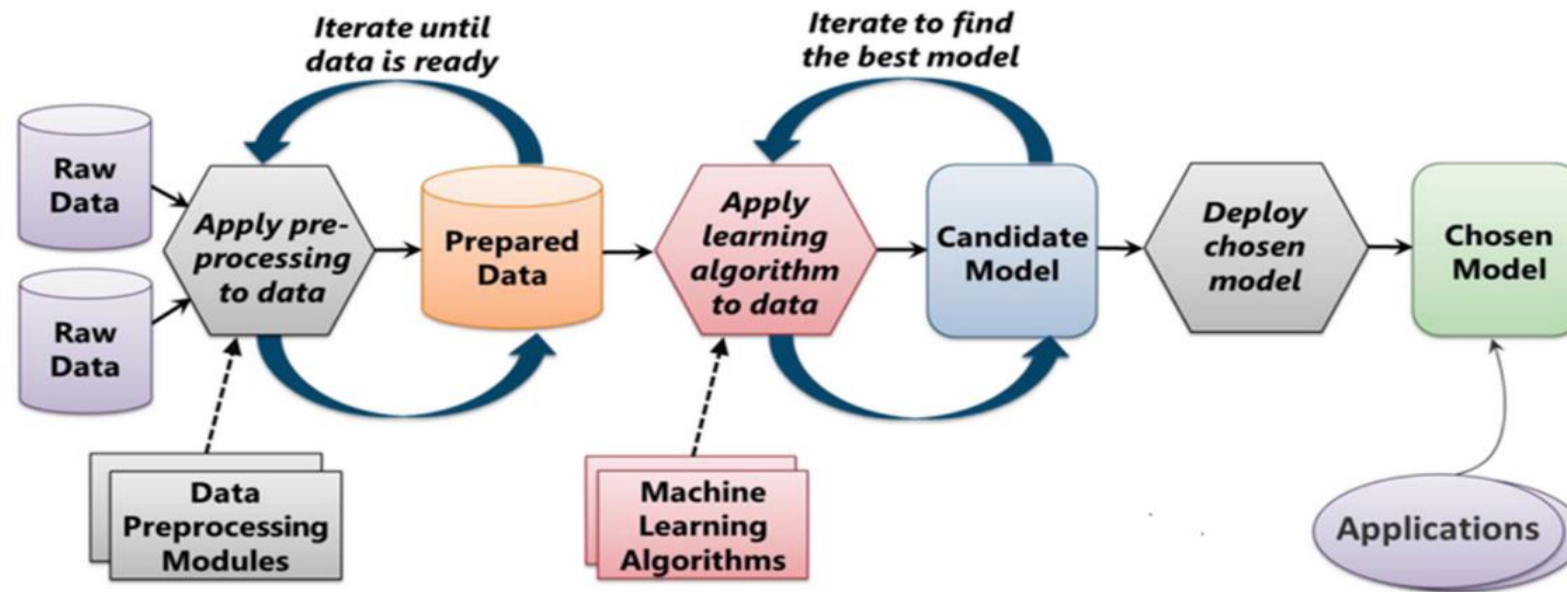
# Learning Objectives

1. Formal Learning Model
2. Definition and Terminology
3. Predictor, Hypothesis, Loss Function
4. Empirical Risk Minimization
5. Probably Approximately Correct
6. Uniform Convergence

# Formal Learning

- Learning is the process of **converting experience** into expertise or **knowledge**.
  - The **input** to a **learning algorithm** is training data, representing experience.
  - The **output** is some **expertise**, which usually takes the form of another computer program that can perform some task.
-

# The Machine Learning Process



Source: D. Chappel, Introducing Azure Machine Learning 2015

# Formal Learning – Stage1 (Inputs/Output)

In ML, there are typically several stages of learning. They might differ from one problem statement to another.

Determine inputs/output. For instance, in **Email spam** detection:

1. **Samples**: Instances of data used for learning
  - Collection of emails categorized as **spam** or **not-spam**
2. **Features**(inputs): The **set of attributes** associated to a sample
  - Length of the message, name of the sender, some characteristics of the header, certain links in the message, the nature of attachments etc.
3. **Labels**(output): Values or Categories assigned to examples (**Spam** or **Not Spam**)

# Formal Learning – Stage2 (Data Size, Model, Training)

- A **learner** (ML model) is **consistent** if it outputs hypotheses that **perfectly fit** the training data, whenever possible.
- Therefore, there must be **sufficient data** to bind the number of examples needed by any consistent learner, to have **all acceptable hypotheses**.
- The **purpose** of learning a model is to use this model **to predict** the label of some unseen examples.
- Before we deploy this ML model, we have to estimate the **quality** of the model's learning.
- If **quality is not good**, we may have to **redo the learning** or in a different way (one reason might be data quality)

# Formal Learning – Stage3 (Dataset)

- **Training Samples:** Examples used to **train** a learning algorithm
  - In our example- a set of emails together with their spam or not spam labels.
- **Validation Samples:** Examples used to **estimate the quality of the learning** so far
  - **Distinct** from the training sample set
  - The **prediction** of the current learning algorithm **is compared with the real value** of the validation samples.
  - Ex: Another set of labeled emails.
- **Test Samples:** Examples used to report the **performance** of the **proposed learning** algorithm (and to give the measure of the expected performance of the model in practice).
  - Distinct from the training and validation sample sets.
  - The prediction of the final learning algorithm is compared with the real value of the validation samples
  - This gives a measure of the accuracy rate of the learning algorithm on new samples.

**Training  
Data**

**Training Split**

**Training  
Samples**

**Validation Split**

**Validation  
Samples**

**Test  
Data**

**Test Samples**

**Test  
Samples**



## Formal Learning – Stage4 (Model Evaluation)

- **Loss Function**: A function that measures the difference (or loss) between a **predicted label** and the **true label** of the sample.
  - In our problem statement: a 0/1 (or true or false) – was the prediction correct?
- **Hypothesis Set**: A set of **parametric functions** mapping features to the set of labels
  - The purpose of the learning algorithm is to **identify the best hypothesis** in this set(based on the available samples), **evaluate its accuracy**, and offer it to be used on unseen samples.
  - **Spam can be a function of**: The originating email domain, its subject, its way of addressing the receiver, its multimedia content, or its hyperlinks.

# Distribution (Terminology)

## **Independently and Identically Distributed(i.i.d):**

In probability theory and statistics, a collection of random variables is independent and identically distributed if each random variable has the same probability distribution as the others and all are mutually independent. This property is usually abbreviated as i.i.d., iid, or IID. In statistics, we usually say “random sample,” but in probability it’s more common to say “IID.” Ex: Flip a Coin-HT

- **Identically Distributed** means that there are no overall trends—the distribution doesn’t fluctuate and all items in the sample are taken from the same probability distribution.
- **Independent** means that the sample items are all independent events. In other words, they aren’t connected to each other in any way. In other words, knowledge of the value of one variable gives no information about the value of the other and vice versa.

## **Distribution (D):**

A distribution is simply a collection of data, or scores, on a variable. Usually, these scores are arranged in order from smallest to largest and then they can be presented graphically.

# Dataset

## Sample:

A **sample** is an unbiased number of **observations taken from a population**.

A sample refers to a **smaller**, manageable version of a larger group of data. It is a subset containing the **characteristics** of a larger population. Samples are used in **statistical testing** when population sizes are too large for the test to include all possible members or observations.

## Features:

- Features, in machine learning, are **variables or attributes** that **describe the characteristics** of a sample or an object.
- They are used as **input** to predictive models, such as regression or classification algorithms, to make predictions.
- Features can be **quantitative** (e.g. numerical) or **qualitative** (e.g. categorical).
- The **choice of features** and how they are represented can have a significant impact on the **performance** of the predictive models.

## Data Points:

- Data points are the **information** we feed into the machine learning model.
- The **number of data points**, or the volume of data, determines the **accuracy** of the model.

# Variables

## Independent variables: $X$ (input)

- The variables that are not affected by the other variables are called independent variables. These are also referred as inputs or Features.
- They are represented as  $X = \{x, x, \dots, x\}$

Examples: For used car price prediction problem – color, brand, mileage, year

## Dependent variable: $Y$ (output)

- We represent  $Y$  as a function of  $X$  and expect  $Y$  to change when the independent variables  $X$  change.
- These are also referred to as output. The variable  $Y$  depends on other variables or factors.

Example: For a used car price prediction problem – Price is output and changes concerning the car features.

# Predictor - Definition

A **Predictor** is a statistical model, machine learning algorithm, or mathematical function used to predict future events or trends based on historical data.

A predictor takes in **input features** and outputs a **predicted value or class**.

The goal of a predictor is to accurately predict the outcome of an event based on a set of input features.

**Predictors** can be used for various applications such as **regression, classification, time series forecasting**, etc.

Examples:

Linear Predictors, Decision Trees, Neural Networks...

# Hypothesis

A **Hypothesis(H)** is an assumption of a result that is falsifiable, meaning it can be proven wrong by some evidence.

A Hypothesis can be either rejected or failed to be rejected.

We never accept any hypothesis in statistics because it is all about probabilities, and we are never 100% certain.

Before the start of the experiment, we define two hypotheses:

1. **Null Hypothesis:** says that there is no significant effect.
2. **Alternative Hypothesis:** says that there is some significant effect

$\hat{y}$  (y-hat:-predicted label/value) and  $y$ (true label/value)

# Hypothesis & Predictor Relation - Example

Does student engagement in academics impact grades? (Problem Statement)

This problem statement can be answered by framing a **Hypothesis**.

Do college students with more study hours achieve a higher GPA when compared to students who don't put in more hours for study? (Predictive statement)

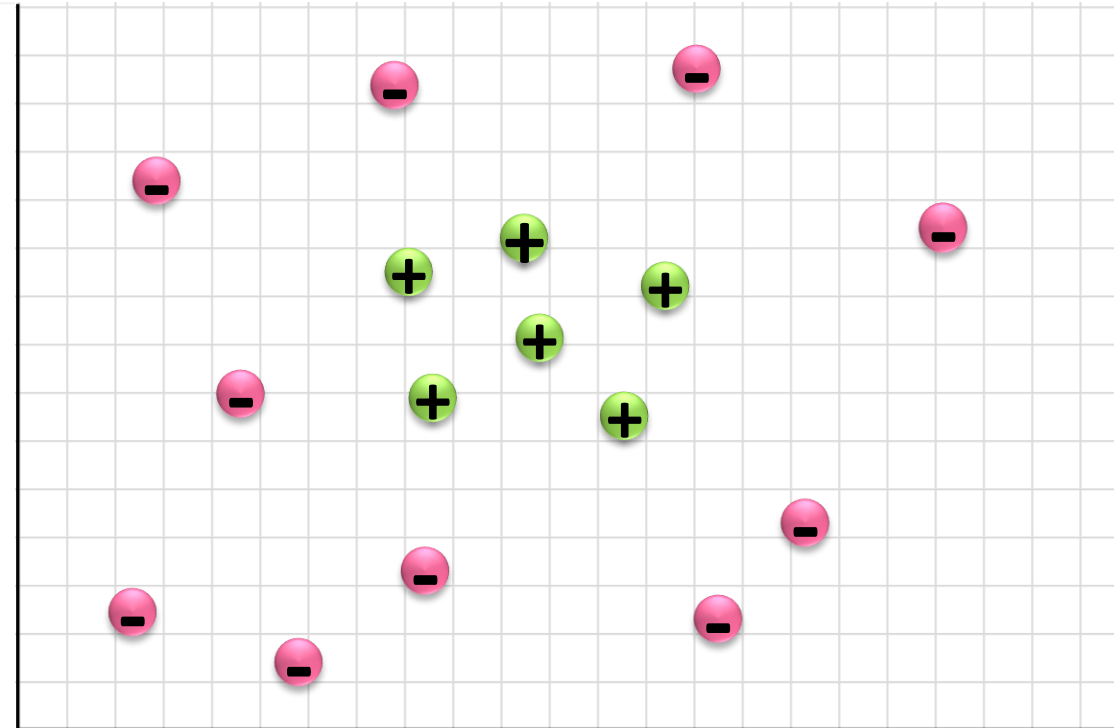
A **Hypothesis(H)** is a predictive statement capable of being tested by a **Predictor** that relates an **independent variable** to the **dependent variable(s)**.

This Hypothesis translates the **problem statement** into a prediction of the expected outcome based on the evidence in the data.

**Null Hypothesis** – Study hours have no impact on GPA.

# Learning A Class From Training Samples

- Example: Determine Class(category) **C** with the classification problem of a “family car”
  - **Prediction**: Is **X** a family car?
  - **Knowledge extraction**: What do people expect from a family car? (Seating Capacity)
- **Input Representation**:  
 $x_1$ : No. of Seats ( $\geq 4$  &  $\leq 8$ )
- **Output**: Positive(+) and Negative(-) examples  
**Y**: Family car/Not Family car





# Training set – X

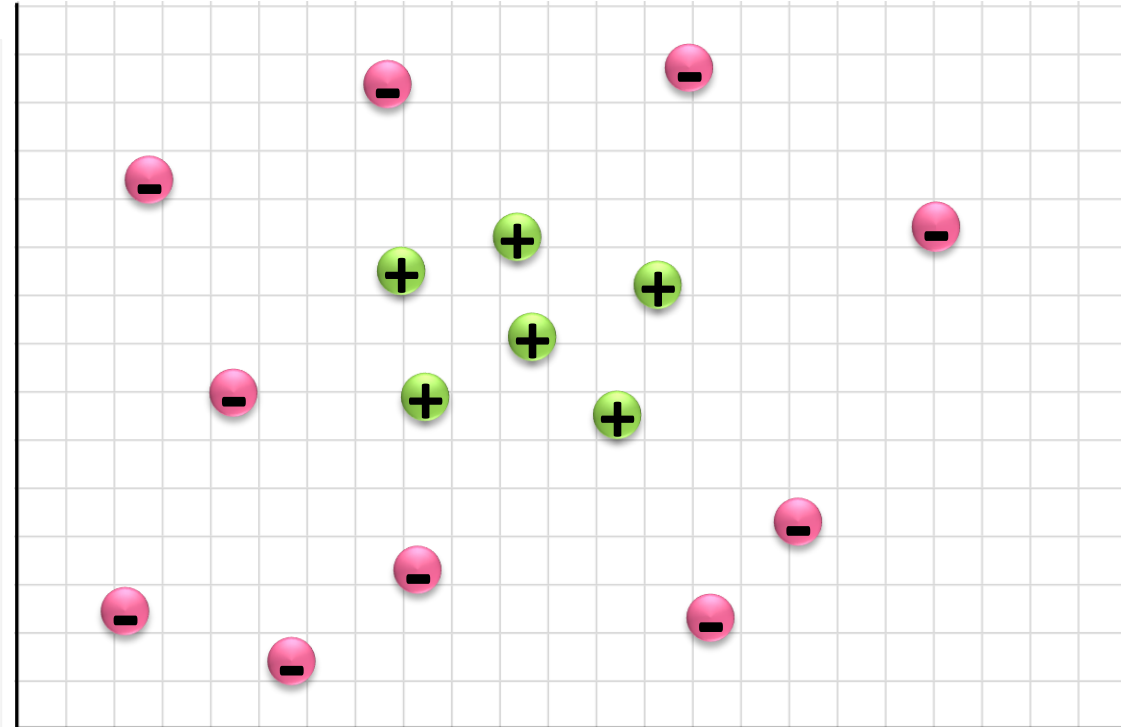
$$X = \{x_1, x_2, x_3 \dots x_N\}$$

$x_1 = \text{No. of Seats}$   
( $\geq 4$  &  $\leq 8$ )

$$X = \{x^t, r^t\}^N$$

t-training set

$$r = \begin{cases} 1 & \text{if } x \text{ is a positive (+) sample} \\ 0 & \text{if } x \text{ is a negative (-) sample} \end{cases}$$



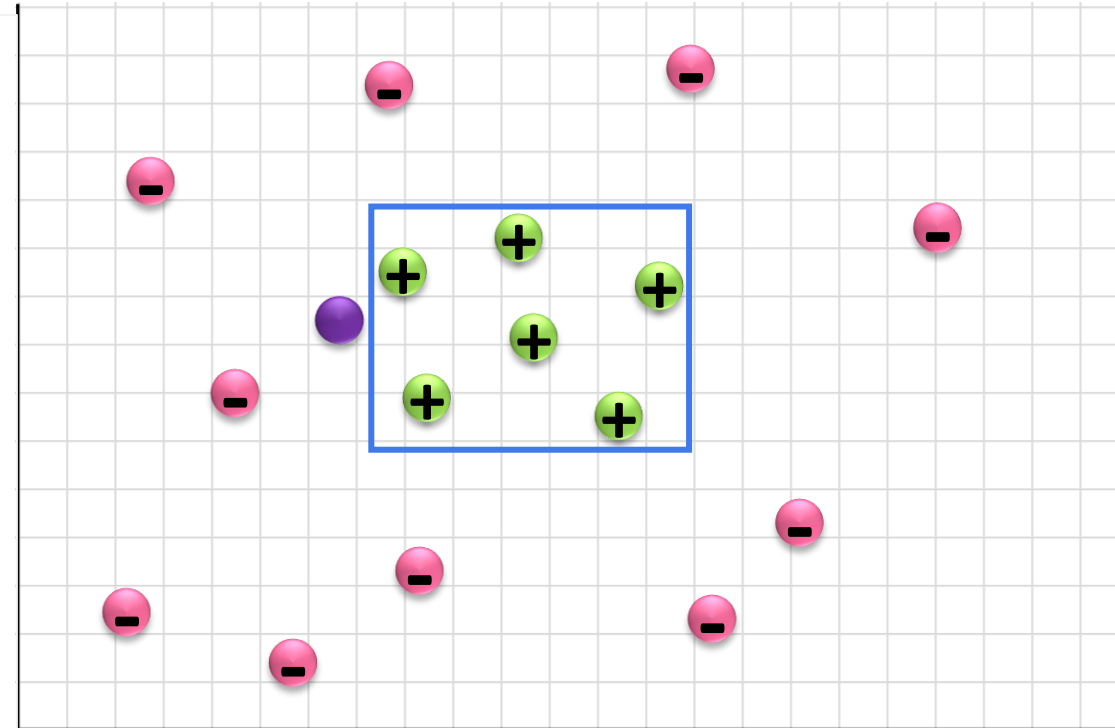
# Hypothesis Class – $H$ (Training set – $X$ )

$$X = \{x_1, x_2, x_3 \dots x_N\}$$

$$X = \{x^t, r^t\}^N$$

$$H = \{h_1, h_2, h_3 \dots\}$$

$$h = \begin{cases} 1 & \text{if } h \text{ classifies } x \text{ is a positive sample} \\ 0 & \text{if } h \text{ classifies } x \text{ is a negative sample} \end{cases}$$

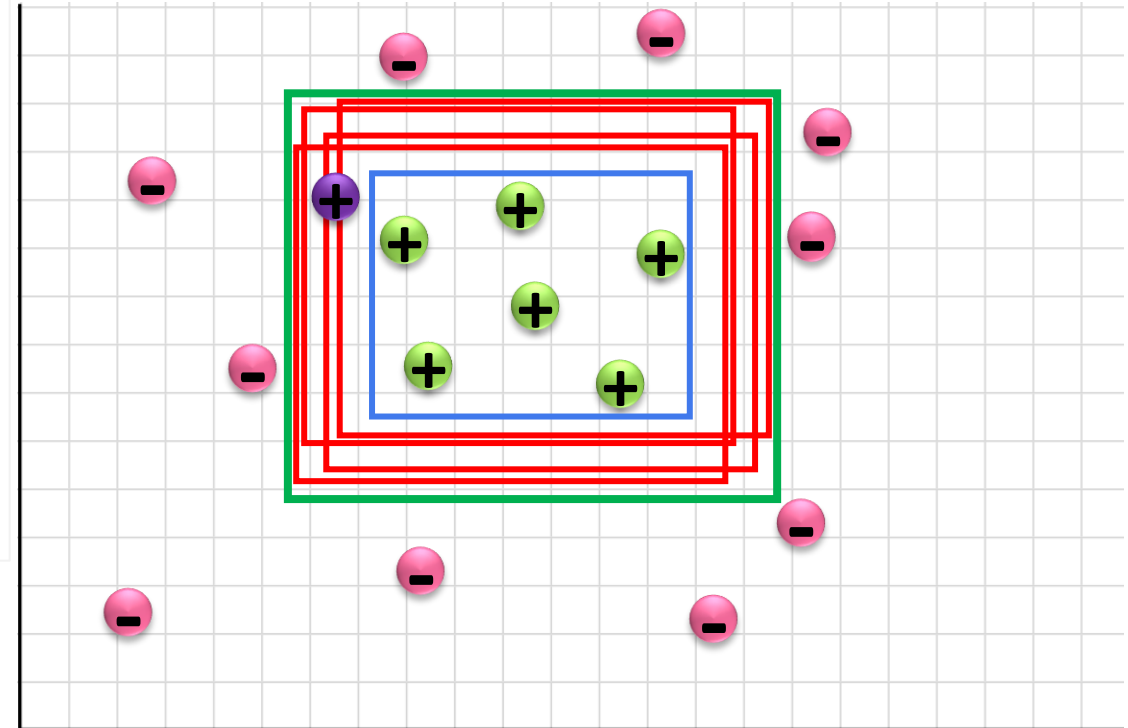


# The most Specific vs General Hypothesis

The most specific Hypothesis **S**

This is the most General Hypothesis **G**

In between **S** and **G**, there is a consistent Hypothesis



# ML Predictor

A Machine Learning Algorithm develops a predictor during the training phase.

Once trained, this predictor can make predictions for unseen or new novel data.

1. Many predictors have the form  $\hat{y} = g(x, \theta)$  { $\hat{y}$ -predicted label,  $g$ -predictor}.
2. This expression can also be written as  $g_{\theta}(x)$ .
3. The function ' $g$ ' determines the structure or form of the predictor
4. Theta ' $\theta$ ' is a set of parameters that can be a vector, matrix, or other data structure. These set of parameters are going to be one of the determinants that produce the output  $\hat{y}$ .
5. The learning is based on the selection of these parameters, while  $g$  is fixed. ( $g$  -Neural Network, Linear Predictor, Decision Trees...)

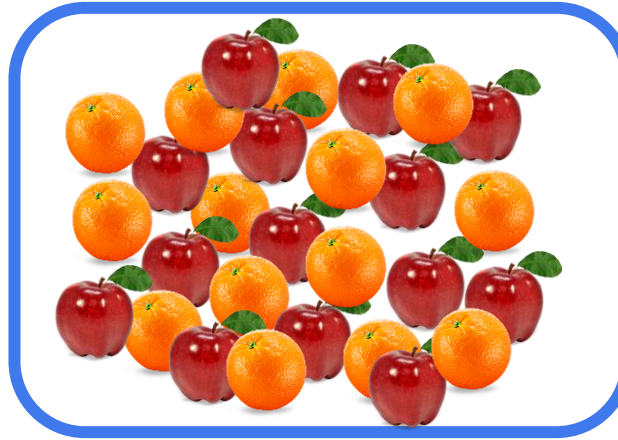
# Sensitivity of Predictor

1. Sensitivity is the factor of How much the Predictor responds to **small changes** in  $X$ (input features).
2. That is, if feature changes are close then their predictions are also close.
3. A Predictor being **insensitive** to **small changes** in data, often refers to a **good attribute**.
4. **Insensitive Predictors often generalize well**, especially in cases when there is less training data.

# Loss Function

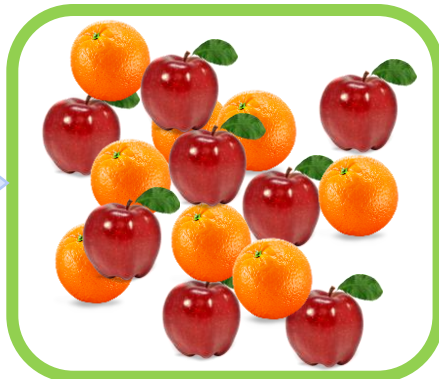
The loss quantifies how well the  $\hat{\mathbf{y}}$  (prediction) approximates  $\mathbf{y}$  (true label).

## DATASET



$\mathbf{Y} = \{\text{APPLE, ORANGE}\}$

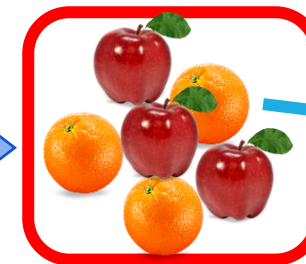
## Training Set



$\mathbf{Y} = \{\text{APPLE, ORANGE}\}$



## Test Set



$\mathbf{Y} = \{\text{ORANGE}\}$

$\hat{\mathbf{y}} = \{\text{APPLE}\}$

# Loss Function

- Loss Function takes 2 vectors as input  $\mathbf{y}$  and  $\hat{\mathbf{y}}$ , and gives a loss function  $l : \mathbf{R}^m \times \mathbf{R}^m \rightarrow \mathbf{R}$
- The loss quantifies how well the  $\hat{\mathbf{y}}$  (prediction) approximates  $\mathbf{y}$  (true label).
- Smaller values of  $l(\hat{\mathbf{y}}, \mathbf{y})$  indicates that  $\hat{\mathbf{y}}$  is a good approximation of  $\mathbf{y}$ , typically  $l(\hat{\mathbf{y}}, \mathbf{y}) \geq 0$

Example:

1. Quadratic Loss:  $l(\hat{\mathbf{y}}, \mathbf{y}) = (\hat{\mathbf{y}} - \mathbf{y})^2$  (for scalar  $y$ ) and  $l(\hat{\mathbf{y}}, \mathbf{y}) = ||\hat{\mathbf{y}} - \mathbf{y}||^2$  (for vector  $y$ )
2. Absolute Loss
3. Fractional Loss or Relative Loss

## Example - Predictor and Loss

1. Linear Regression Model for scalar  $\mathbf{y}$ ,

$$\hat{\mathbf{y}} = \mathbf{g}_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}_1 \mathbf{x}_1 + \boldsymbol{\theta}_2 \mathbf{x}_2 + \dots + \boldsymbol{\theta}_d \mathbf{x}_d$$

Here  $\boldsymbol{\theta} \in \mathbb{R}^d$  is a vector {d is dimensional vector}

$$\text{Scalar Loss } l(\hat{\mathbf{y}}, \mathbf{y}) = (\hat{\mathbf{y}} - \mathbf{y})^2$$

2. Linear Regression Model for vector  $\mathbf{y}$ ,

$$\hat{\mathbf{y}} = \mathbf{g}_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}_1 \mathbf{x}_1 + \boldsymbol{\theta}_2 \mathbf{x}_2 + \dots + \boldsymbol{\theta}_d \mathbf{x}_d$$

Here  $\boldsymbol{\theta}$  is a collection of m-vectors  $\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_d \in \mathbb{R}^m$

Usually organized as (d X m) matrix  $\boldsymbol{\theta}$  with rows  $\boldsymbol{\theta}_i$  Transpose

$$\text{Vector Loss } l(\hat{\mathbf{y}}, \mathbf{y}) = ||\hat{\mathbf{y}} - \mathbf{y}||^2$$



# Empirical Risk

1. The Empirical Risk is the average loss over the data points.

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n l(\hat{y}^i - y^i)$$

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n l(\mathbf{g}_{\boldsymbol{\theta}}(\mathbf{x})^i - y^i)$$

2. If  $\mathcal{L}(\boldsymbol{\theta})$  is small, the predictor **predicts** or fits the given data **well**.

3. Empirical risk is used to train a **parameterized prediction** model.

# Empirical Risk Minimization (ERM)

1. Empirical Risk Minimization (**ERM**) is the process by which Predictors learn from data.
2. **ERM** is a general method for choosing  $\Theta$  i.e., fitting a parameterized predictor.
3. **ERM** Chooses  $\Theta$  to minimize the **Empirical Risk**  $\mathcal{L}(\Theta)$ .
4. **ERM** chooses  $\Theta$  by **attempting to match the given dataset well**, as measured by the loss  $l$  such that the **average loss of the entire dataset is small**.
5. The predictor found by ERM depends on the loss we choose. Ex: **Mean Squared Error, Mean Absolute Error, Fractional Error**.
6. Finally, we validate this predictor with external Test data using performance metrics.

## ERM - Concern

1. One of the **empirical-minimum-cost hypotheses** is that no classifier can have smaller(0) error.
2. That is the loss **L cannot be 0**.
3. We have found a predictor whose performance on the **training set is excellent**, yet its performance on the **true “world” is very poor**.
4. This phenomenon is called **overfitting**.
5. Intuitively, overfitting occurs when our **hypothesis fits the training data “too well”**

# ERM – Inductive Bias

1. A common solution is to apply the **ERM learning rule** over a restricted search space.
2. Formally, **the learner should choose in advance** (before seeing the data) **a set of predictors**. This set is called a **hypothesis class** and is denoted by  $H$ .
3. Each  $h \in H$  is a function mapping from  $X$  to  $Y$ .
4. For a given class  $H$ , and a training sample,  $S$ , the  $ERM_H$  learner uses the **ERM rule to choose a predictor  $h \in H$** .
5. By restricting the learner to choosing a predictor from  $H$ , we bias it toward a particular set of predictors.
6. Such restrictions are often called **inductive bias**.

# Finite Hypothesis Classes

1. Since the choice of such a restriction is determined before the learner sees the training data, it should ideally be based on some prior knowledge about the problem to be learned.
2. The simplest type of restriction on a class is imposing an upper bound on its size (that is, the number of predictors  $h$  in  $H$ )
3. If  $H$  is a finite class, then ERMH will not overfit, provided it is based on a sufficiently large training sample
4. The ERMH rule over a finite hypothesis class will be probably (with confidence  $1-\delta$ ) approximately (up to an error of  $\epsilon$ ) correct.

# Probably Approximately Correct

When we design Learning Algorithms, there are several fundamental questions to consider, like

1. What can be learned efficiently? – Suppose a dataset is given, how much time is required by the Algorithm to learn about the dataset
2. What problems are inherently hard to learn in the dataset? – For some hard problems an Algorithm may not solve them in efficient time, no matter how large the given dataset is
3. How many samples are needed for learning successfully?

To address such questions we need a Formal framework that connects Machine Learning to Computational Complexity.

One such Framework is Probably Approximately Correct (PAC) – Introduced by L.Valiant in 1984.

# PAC Learning

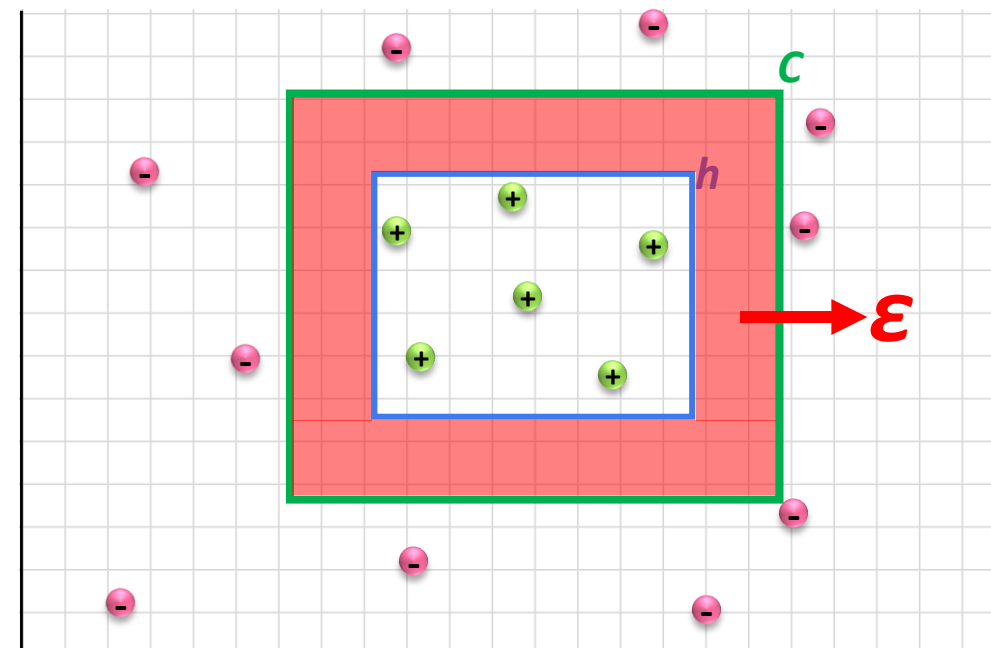
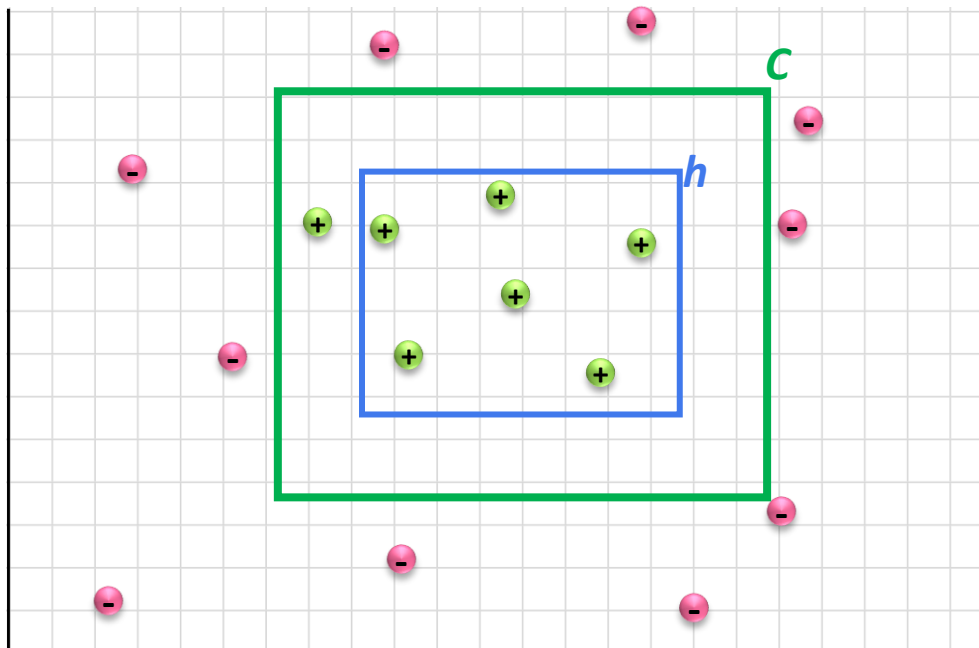
- The probably approximately correct (PAC) model considers algorithms that learn target concepts from some concept class  $\mathbf{C}$ , using training examples drawn at random according to an unknown, but fixed, probability distribution  $\mathbf{D}$ .
- One constraining assumption of the PAC learning model is that the learner knows in advance some restricted concept class  $\mathbf{C}$  that contains the target concept to be learned.
- In contrast, the agnostic learning model considers the more general setting in which the learner makes no assumption about the class from which the target concept is drawn.
- Instead, the learner outputs the hypothesis from  $\mathbf{H}$  that has the least error (possibly nonzero) over the training data.

# PAC Learning

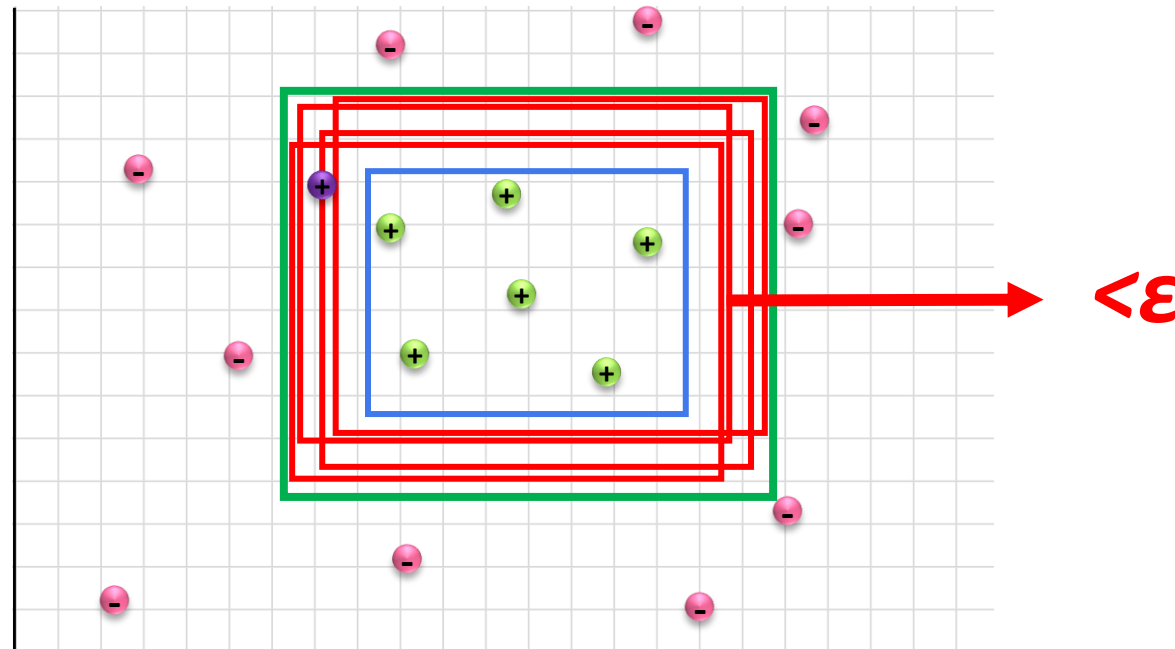
- Much early work on computational learning theory dealt with the question of whether the learner could identify the target concept in the limit, given an indefinitely long sequence of training examples.
- Current research on computational learning theory covers a broad range of learning models and learning algorithms.
- For the Learning problem we will now consider the PAC learning model.
- How many training examples and how much computations are required in order to learn various classes of target functions within the PAC model



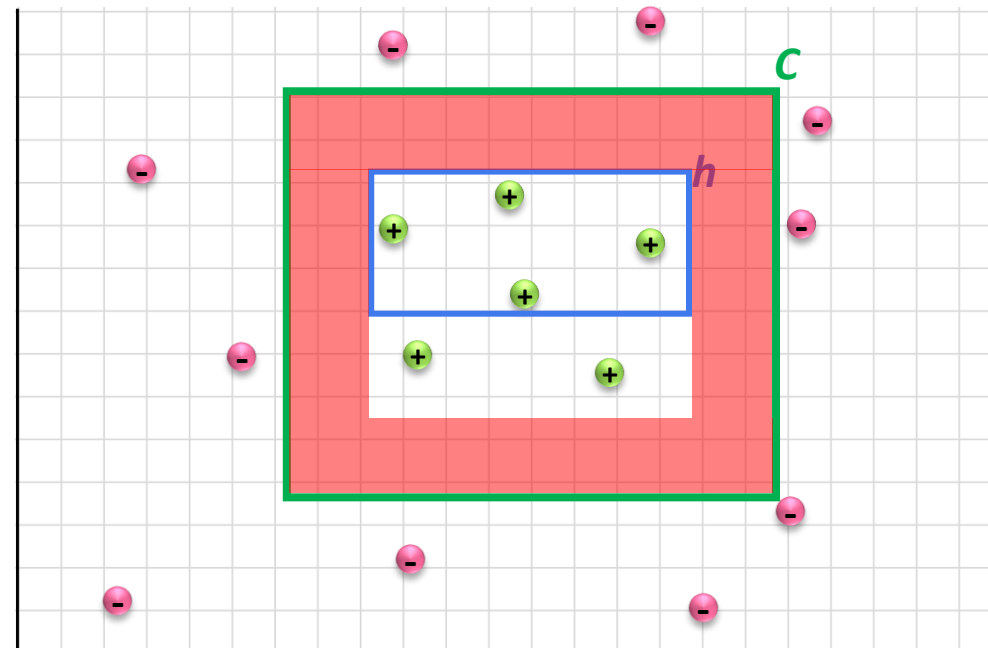
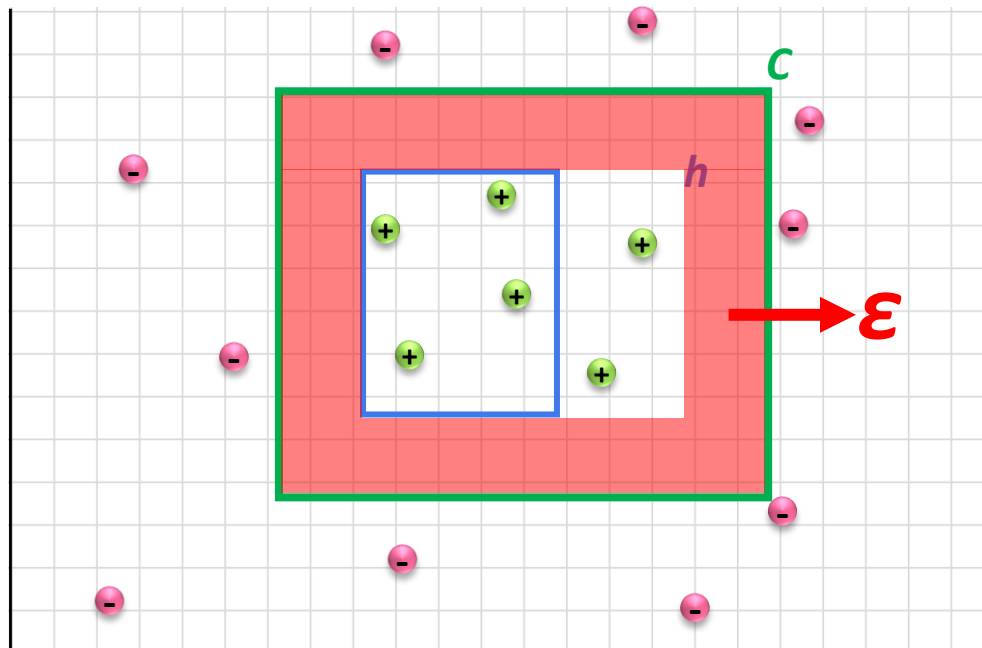
# PAC Learning



# PAC Learning



# PAC Learning



$> \epsilon$

# PAC Learning

| ERROR (LOSS) |
|--------------|
| 0.025        |
| 0.049        |
| 0.051        |
| 0.032        |
| 0.025        |
| 0.068        |
| 0.041        |
| 0.037        |
| 0.019        |
| 0.005        |

$$\epsilon = 0.05$$

*No. of samples = 10*

*No. of errors( $> \epsilon$ ) = 2*

The Hypothesis is Probably Approximately Correct for 8 times and failed for 2 times

$$P(H) = 8/10 = 0.8$$

$$P(H_1) = 0.78 \quad P(H_2) = 0.82 \quad P(H_3) = 0.90 \quad P(H_4) = 0.80$$

# Probably Approximately correct (PAC) Learning

- Let ' $\mathbf{X}$ ' refer to the set of all possible instances over which target functions may be defined.
- $\mathbf{X}$  might represent the set of all people, each described by the attributes age (e.g., young or old) and height (short or tall)
- Let  $\mathbf{C}$  refer to some set of target concepts that our learner might be called upon to learn. Each target concept  $\mathbf{c}$  in  $\mathbf{C}$  corresponds to some subset of  $\mathbf{X}$ , or equivalently to some boolean-valued function  $\mathbf{c} : \mathbf{X} \rightarrow \{0, 1\}$
- For example, one target concept  $\mathbf{c}$  in  $\mathbf{C}$  might be the concept "people who drink coffee".
- If  $\mathbf{x}$  is a positive example of  $\mathbf{c}$ , then we will write  $\mathbf{c}(\mathbf{x}) = 1$ ; if  $\mathbf{x}$  is a negative example,  $\mathbf{c}(\mathbf{x}) = 0$ .
- We assume instances are generated at random from  $\mathbf{X}$  according to some probability distribution  $\mathcal{D}$  ( $\mathcal{D}$  maybe any distribution)

# PAC Learning

| ERROR (LOSS) |
|--------------|
| 0.025        |
| 0.049        |
| 0.051        |
| 0.032        |
| 0.025        |
| 0.068        |
| 0.041        |
| 0.037        |
| 0.019        |
| 0.005        |

$$\epsilon = 0.05 \quad \delta = 0.20$$

$$P(H1) = 8/10 = 0.8$$

$$1 - \delta = 1 - 0.20 = 0.80$$

$$P(H1) \geq 1 - \delta$$

$$P(H2) = 0.78 \quad P(H2) < 1 - \delta$$

# Probably Approximately correct (PAC) Learning

- Training examples are generated by drawing an instance  $x$  at random according to  $\mathcal{D}$ , then presenting  $x$  along with its target value,  $c(x)$ , to the learner.
  - **Learner** considers some set  $H$  of possible hypotheses when attempting to learn the target concept.
  - A sequence of training examples of the target concept  $c$ , **Learner** must output some hypothesis  $h$  from  $H$ , which is its estimate of  $c$
-

# Probably Approximately correct (PAC) learning

- A hypothesis class  $\mathbf{H}$  is PAC learnable if there exists a function  $\mathbf{m}_{\mathbf{H}}:(0,1)^2 \rightarrow \mathbf{N}$  and a learning algorithm with the following property:
  - For every  $\epsilon, \delta \in (0,1)$ , for every distribution  $\mathbf{D}$  over  $\mathbf{X}$ , and for every labeling function  $\mathbf{f}:\mathbf{X} \rightarrow \{0, 1\}$ , if the realizable assumption holds with respect to  $\mathbf{H}$ ,  $\mathbf{D}$ ,  $\mathbf{f}$ , then when running the learning algorithm on  $\mathbf{m} \geq \mathbf{m}_{\mathbf{H}}(\epsilon, \delta)$  i.i.d. examples generated by  $\mathbf{D}$  and labeled by  $\mathbf{f}$ , the algorithm returns a hypothesis  $h$  such that, with the probability of at least  $1-\delta$  (over the choice of the examples),  $L_{\mathbf{D}, \mathbf{f}}(h) \leq \epsilon$
- $\epsilon$  - accuracy parameter (how far the output classifier is from the optimal) - Approximately.
- $\delta$  - confidence parameter (how likely the classifier is to meet the accuracy requirement) - Probably.
- $\mathbf{m}_{\mathbf{H}}:(0,1)^2 \rightarrow \mathbf{N}$  defines sample complexity (how many samples are needed).



## Some Other Learning Tasks

1. In a Multi-Class prediction problem, the set of labels  $Y$  could be larger than just  $(0,1)$ .
  2. Example: Title of a document – topics such as Sports, Tech, Business, Health, Politics, Entertainment....
  3. How can we define success of a predictor here?
  4. Loss can be very general. With Classification, error labels can be penalized.
-

# PAC Summary

1. A function is learnable if there exists an algorithm with a **high probability** that when this algorithm trains on a randomly selected training set, we get a **good generalization error**.
2. There are two important goals of PAC learnability:
  - **Low generalization Error**
  - **High probability** of achieving that **low generalization error**.
3. To achieve both the goals, we need a **reasonable sample size** for the PAC learnability.
4. **Smaller** values of  $\epsilon$  and  $\delta$  will correspond to better learning.

## General Learning Models

1. The model we have just described can be readily generalized so that it can be made relevant to a wider scope of learning tasks.
2. The learning task that we have been discussing so far has to do with predicting a binary label to the given example (like being a family car or not).
3. However, many learning tasks take a different form. Like **Regression** instead of **Classification**.
4. For example, one may wish to predict a real-valued number (say, the temperature at 9:00 a.m. tomorrow).
5. It turns out that our analysis of learning can be readily extended to such and many other scenarios by allowing a variety of loss functions.

# More Generality for Learning

Case1: Binary Label Prediction

$$\mathbf{Z} = \mathbf{X} \times \{0,1\},$$

$$l(h(\mathbf{x}, \mathbf{y})) = \begin{cases} 1 & \text{if } h(\mathbf{x}) = \mathbf{y} \\ 0 & \text{if } h(\mathbf{x}) \neq \mathbf{y} \end{cases}$$

Case2: Multiclass Prediction

$$\mathbf{Z} = \mathbf{X} \times \mathbf{Y},$$

$$l(h(\mathbf{x}, \mathbf{y})) = \begin{cases} 1 & \text{if } h(\mathbf{x}) = \mathbf{y} \\ 0 & \text{if } h(\mathbf{x}) \neq \mathbf{y} \end{cases}$$

Case3: Regression

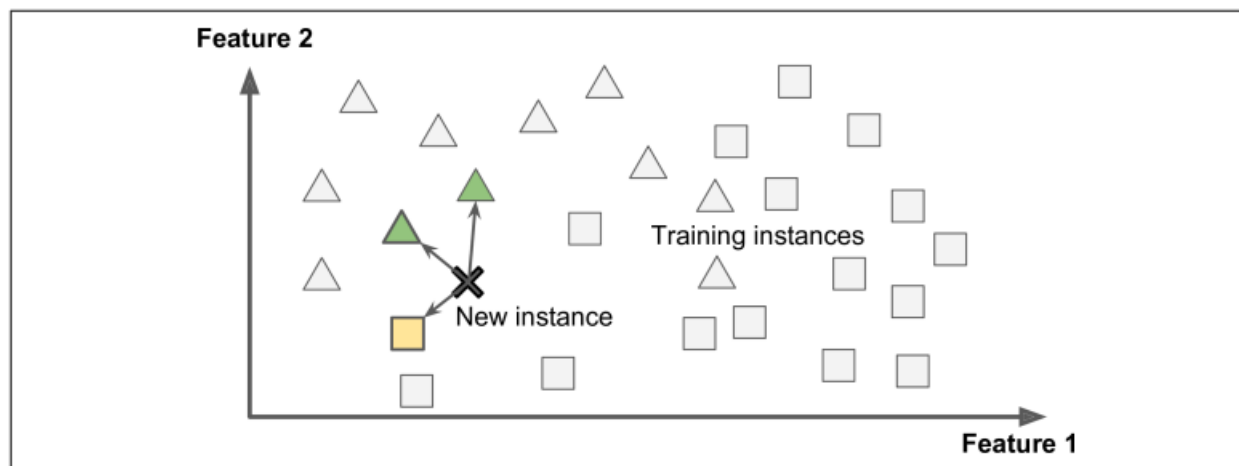
$$\mathbf{Z} = \mathbf{X} \times \mathbf{R},$$

$$l(h(\mathbf{x}, t)) = (h(\mathbf{x}) - t)^2 \text{ Square loss}$$

# Instance-Based Learning

**Instance-based learning:** The system learns the examples by heart, then generalizes to new cases by comparing them to the learned examples, using a similarity measure.

Example: An Instance-based spam filter will mark all new emails as Spam if they are identical to emails already flagged by users as Spam.

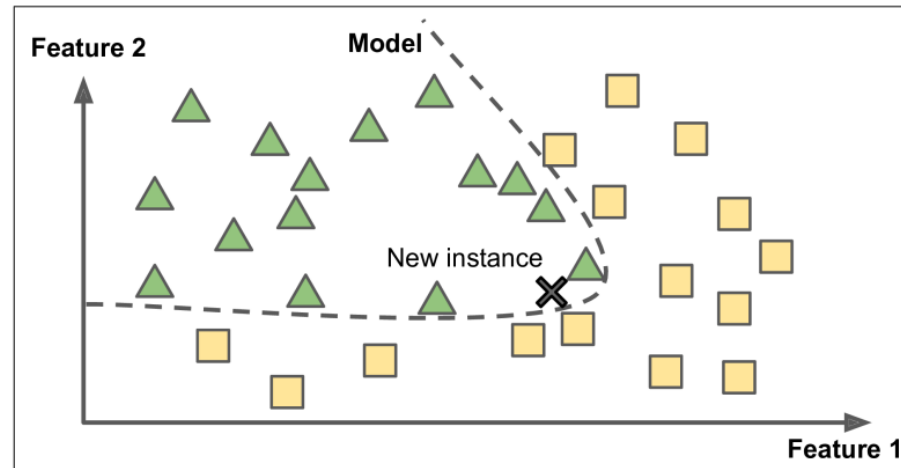


In the picture the new instance **X** would be classified as a triangle because the majority of the most similar instances belong to that class.

# Model-Based Learning

Another way to generalize from a set of examples is to build a model of these examples, then use that model to make predictions. This is called model-based learning

Example: A model learns the patterns of a Spam email based on training using the training data, to predict if a new email is Spam or not.



The new instance would be classified as a triangle because the model separates the triangle from a square with a boundary drawn using training.

## Learning via Uniform Convergence

Upon receiving a training sample,  $S$ , a learner evaluates the risk (or error) of each  $h$  in  $H$  on the given sample.

Then it outputs a member of  $H$  that minimizes this empirical risk. (best  $h$ )

The hope is that an  $h$  that **minimizes the empirical risk** with respect to  $S$  is a **risk minimizer** with respect to the true data probability distribution as well.

Whenever the sample is  $(\epsilon/2)$ -representative, the ERM learning rule is guaranteed to return a good hypothesis.

Uniformly over all hypotheses in the hypothesis class, the empirical risk will be close to the true risk.

## Learning via Uniform Convergence

### Definition:

A sample  $S=(x_1,y_1)....(x_m,y_m)$  is epsilon representative of a class  $H$  with respect to a Distribution on  $D$ :

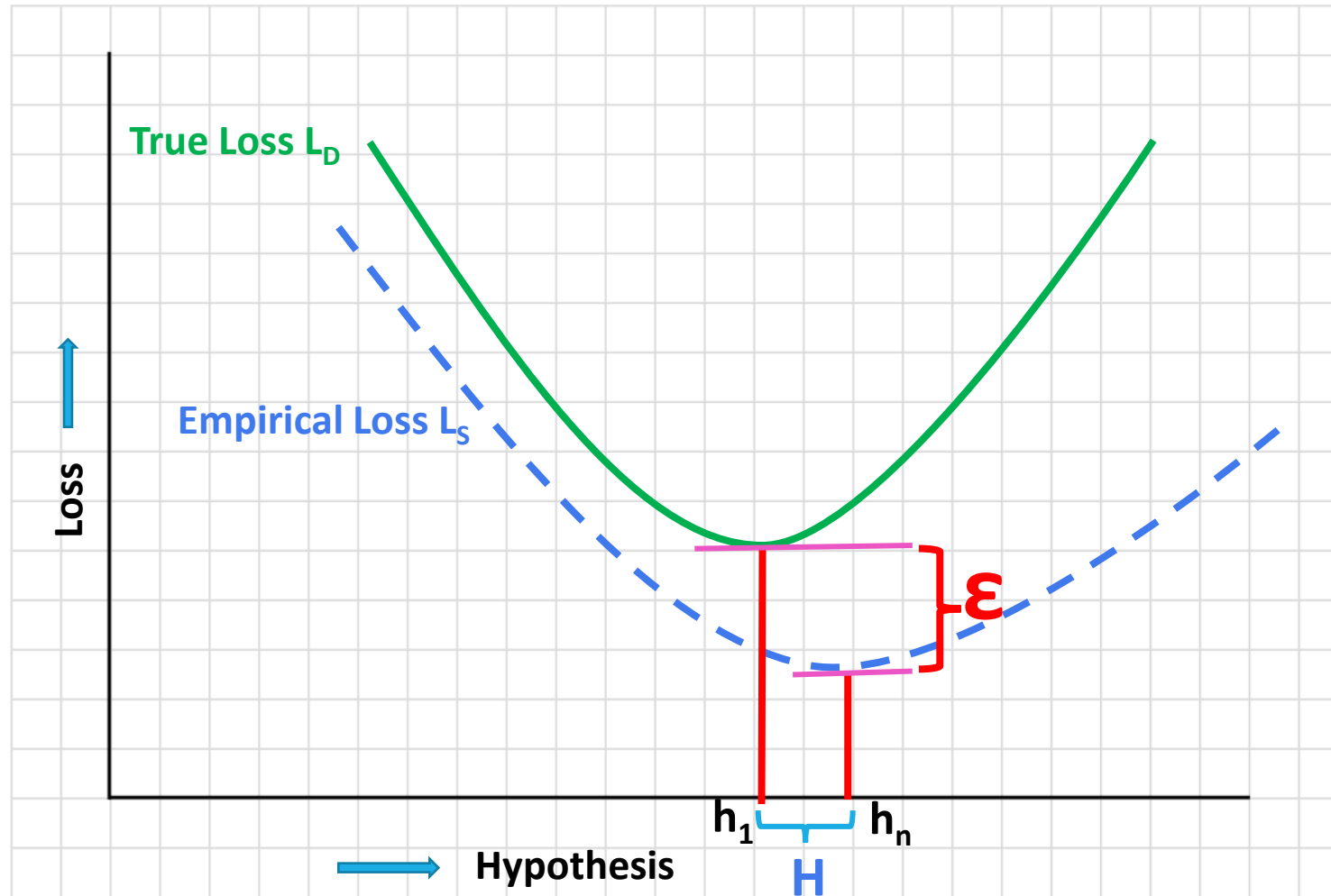
If for every  $h$  in  $H$ , the loss of  $h$  with respect to the sample minus the loss of  $h$  with respect to the distribution is at most epsilon  $\epsilon$ .

$$L_S(h) - L_D(h) \leq \epsilon$$

Where  $L_S(h) = 1/m \sum_{z \in S} l(h, z)$



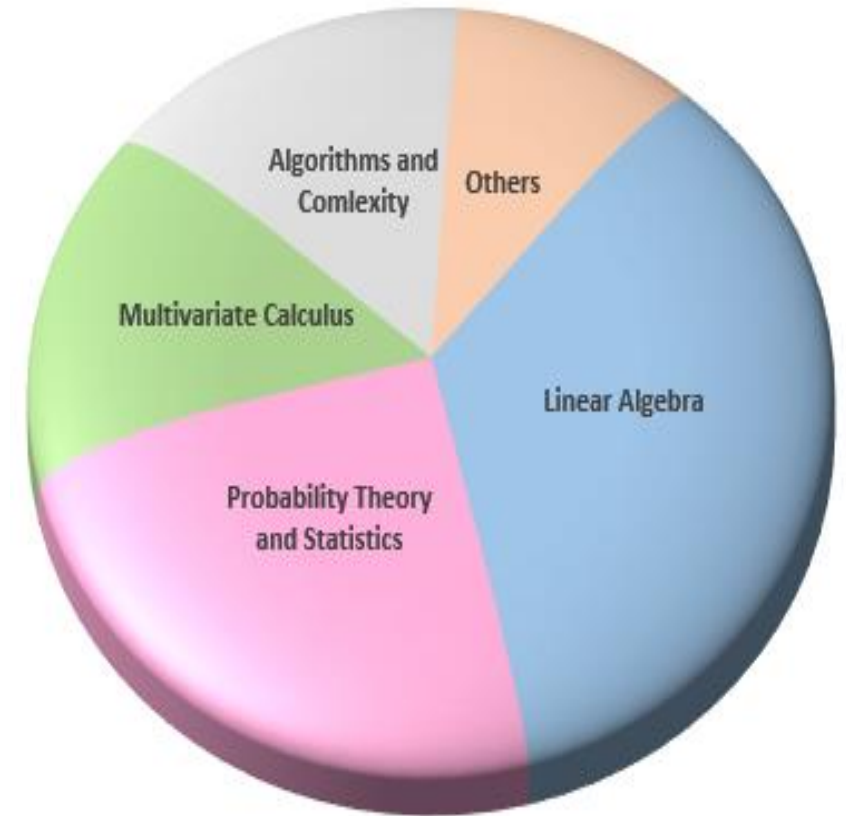
# Illustration - Learning via Uniform Convergence



# Mathematics for ML



- Linear Algebra:  
<https://www.khanacademy.org/math/linear-algebra>
- Probability and Statistics:  
<https://www.khanacademy.org/math/probability>
- Multivariate Calculus:  
<https://www.khanacademy.org/math/multivariable-calculus>
- Optimization:  
<https://www.khanacademy.org/math/ap-calculus-ab/ab-diff-analytical-applications-new/ab-5-11/e/optimization>



**END**

---